SIMATS ENGINEERING

ASSESSMENT TOOL 2 — DATA INTERPRETATION (MEDIUM)

Course: Internet Programming **Course Code:** CSA43

Name: P. Revanth Reddy **Register Number:** _____

Total Marks: 20

1. HTTP Response Analysis

1) Incorrect or invalid fields: The `Content-Length: -150` field is invalid because Content-Length must be a non-negative decimal number representing the message-body size in bytes. The Date line also has a weekday mismatch: 1 April 2026 is a Wednesday, not Monday. The remaining fields shown are syntactically plausible.

2) Issue with Content-Length and its impact: A negative length cannot describe an HTTP message body. It can cause the response to be rejected or handled as malformed by a server, proxy, or client, and may interfere with message framing. The value must equal the actual body length in bytes if Content-Length is used.

3) Does status code 200 match response correctness? `200 OK` means the request succeeded; it does not guarantee that every header is valid. Therefore, the status code alone does not make this malformed response correct. The server should send a valid response with consistent headers.

4) Corrections: Replace the negative length with the actual non-negative body size, for example `Content-Length: 150` only if the body is exactly 150 bytes. Correct the date to `Date: Wed, 01 Apr 2026 10:00:00 GMT` if that timestamp is intended. Keep `Content-Type: text/html` when the body is HTML. Alternatively, use valid transfer framing appropriate to the HTTP version.

2. DOM Structure Analysis

1) Missing or improperly closed elements: The <p> element containing “HelloWorld” is not explicitly closed before the `</div>` tag. The intended nesting is therefore malformed in the supplied markup.

2) Browser interpretation: HTML browsers use error-recovery rules to build a DOM even from imperfect markup. The browser will generally infer the end of the paragraph when it encounters the closing `</div>`, then close the div and continue parsing. Exact DOM repair can depend on the parsing rules and surrounding markup.

3) Impact on rendering and layout: The text may still appear, because browsers repair many common HTML errors. However, unexpected element nesting can affect CSS selectors, spacing, accessibility tools, and JavaScript that expects a particular DOM structure. Relying on automatic repair makes the document harder to maintain.

4) Corrected markup:

```
<!DOCTYPE html>
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld</p>
    </div>
  </body>
</html>
```

3. GET vs POST in Login System

1) Difference in data transmission: With GET, form data is commonly placed in the URL query string, such as `/login?user=admin&pass=1234`. URLs can be stored in browser history, server logs, analytics, and referrer data. With POST, submitted fields are normally carried in the request body, not displayed in the URL. POST therefore avoids exposing credentials in the URL, but it does not encrypt them by itself.

2) More suitable method: POST is more suitable for submitting login credentials because credentials are not placed in the URL. The login request must also use HTTPS, and the server must validate the credentials securely.

3) Security risks of GET: Usernames and passwords in URLs may be saved in browser history, copied or shared, captured in logs, or exposed through monitoring and referrer information. This increases the risk of credential disclosure. GET requests may also be cached or bookmarked in unsuitable implementations.

4) Best practices: Use HTTPS/TLS; submit credentials using POST; never log passwords; store passwords using a strong salted password-hashing algorithm; validate input on the server; apply rate limiting and protections against brute-force attacks; use secure session cookies (Secure, HttpOnly, and appropriate SameSite settings); and use multi-factor authentication where possible. POST alone is not a substitute for HTTPS.

4. HTML Code Inspection

1) Missing closing tags: The `<h1>` and `<p>` elements are not explicitly closed. Although browsers can infer certain end tags, explicitly closing them improves correctness and readability.

2) Issues with the img tag: The `<img src="image.jpg">` element has no `alt` attribute. Alternative text should describe the image for screen-reader users and when the image cannot load; use `alt=""` if the image is purely decorative. The `img` element is a void element in HTML, so it does not require a closing tag.

3) Browser handling: The browser will usually infer the missing heading and paragraph end tags and still render the page. If `image.jpg` cannot be found, the image will not display, and the missing alternative text leaves users without a text equivalent.

4) Corrected code:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Sample</title>
  </head>
  <body>
    <h1>Welcome</h1>
    <p>This is a paragraph.</p>
    <img src="image.jpg" alt="Description of the image">
  </body>
</html>
```

Replace “Description of the image” with an accurate description of the actual image.

5. Browser–Server Communication Flow

1) Explanation of each step:
1. **User enters a URL:** The browser reads the scheme, hostname, path, and any query parameters.
2. **DNS lookup:** The browser or operating system resolves the hostname to an IP address, often using a cached result.
3. **HTTP request:** The browser establishes the required connection (TCP and, for HTTPS, TLS; HTTP/3 uses QUIC) and sends the request.
4. **Server processing:** The server routes the request, runs application logic if needed, and obtains or generates the requested resource.
5. **Response returned:** The server sends a status code, headers, and usually a response body.
6. **Browser renders page:** The browser parses HTML, builds the DOM, processes CSS, executes JavaScript as needed, and fetches additional resources.

2) Where delays can occur: Delays may arise from DNS resolution, connection setup, TLS negotiation, network latency or packet loss, server queueing and processing, database calls, large response bodies, and downloading/rendering images, CSS, JavaScript, or fonts. Client-side JavaScript can also delay rendering.

3) Role of DNS: DNS translates a human-readable hostname (such as `example.com`) into an IP address that the client can connect to. DNS does not normally transfer the webpage itself. Caching DNS answers can reduce repeated lookup time.

4) Performance optimizations: Use DNS caching and reliable DNS service; enable keep-alive and modern HTTP versions; use HTTPS efficiently; compress text responses with Brotli or gzip; optimize and appropriately size

images; minimize unnecessary JavaScript and CSS; cache static resources with suitable cache headers; use a CDN for geographically distributed users; reduce server/database processing time; and defer non-critical scripts or lazy-load below-the-fold images.

Conclusion: Correct HTTP headers, valid HTML structure, secure authentication practices, and efficient browser–server communication improve reliability, accessibility, security, and page-loading performance.

Student declaration: I certify that I have reviewed these answers and will ensure that my submitted work follows the assessment's academic-integrity requirements.

Signature: _____